Assignment 3 Group 10 - Magnification 100

In [1]:
```python
import tarfile
import pandas as pd
import os
```

In [2]:
```python
def count_png_files_in_folder(folder_path):
    png_count = 0

    for root, dirs, files in os.walk(folder_path):
        png_count += sum(1 for file in files if file.endswith('.png'))
    return png_count

count_png_files_in_folder("/Users/renkasik/Desktop/Intro To Focus Areas/data/BreaKHis_v1")
```

Out[2]:
```
7909
```

Import data to df with Benign/Malignant IDs and choosing magnification

In [3]:
```python
def extract_image_labels(folder_path, magnification_lvl):
    file_paths = []
    labels = []

    for root, dirs, files in os.walk(folder_path):
        for file_name in files:
            if file_name.endswith('.png'):
                file_path = os.path.join(root, file_name)
                # Extract parts of the filename
                parts = file_name.split('-')

                label = parts[0].split('_')[1]  # 'M' or 'B' indicating malignancy
                magnification = parts[3]  # Magnification levels

                # Filter for images with 100x magnification
                if magnification == str(magnification_lvl):
                    labels.append(label)
                    file_paths.append(file_path)

    # Create a DataFrame with the extracted data
    df = pd.DataFrame({
        'FilePath': file_paths,
        'Label': labels
    })

    return df
```

In [4]:
```python
image_labels_df = extract_image_labels("/Users/renkasik/Desktop/Intro To Focus Areas/data/BreaKHis_v1", 100)
```

In [5]:
```python
image_labels_df.head(5)
```

Out[5]:

	FilePath	Label
0	/Users/renkasik/Desktop/Intro To Focus Areas/d...	M
1	/Users/renkasik/Desktop/Intro To Focus Areas/d...	M
2	/Users/renkasik/Desktop/Intro To Focus Areas/d...	M
3	/Users/renkasik/Desktop/Intro To Focus Areas/d...	M
4	/Users/renkasik/Desktop/Intro To Focus Areas/d...	M

Split data into train. test and validation sets

In [6]:
```python
from sklearn.model_selection import train_test_split
```

In [7]:
```python
labels = image_labels_df['Label']

X_train, X_temp = train_test_split(image_labels_df, test_size=0.3, stratify=labels, random_state = 42)

label_test_val = X_temp['Label']

X_test, X_val = train_test_split(X_temp, test_size=0.5, stratify=label_test_val, random_state = 42)

print('The shape of train data',X_train.shape)
print('The shape of test data',X_test.shape)
print('The shape of validation data',X_val.shape)
```

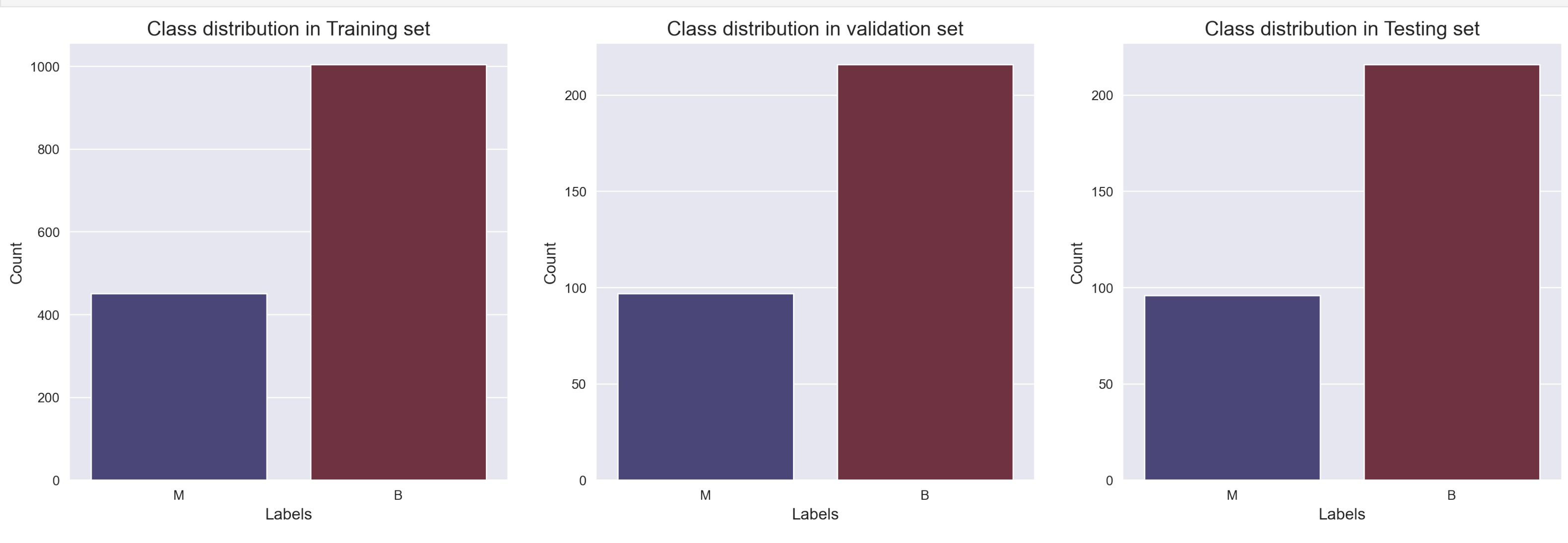
```
The shape of train data (1456, 2)
The shape of test data (312, 2)
The shape of validation data (313, 2)
```

In [8]:
```python
# Convert Labels to Numeric
label_mapping = {'M': 1, 'B': 0}
X_train['Label'] = X_train['Label'].map(label_mapping)
X_val['Label'] = X_val['Label'].map(label_mapping)
X_test['Label'] = X_test['Label'].map(label_mapping)
```

Distribution of train, test and validation Datasets

In [9]:
```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [10]:
```python
labels = ['M','B']

#count unique label appearances
label1,count1 = np.unique(X_train.Label,return_counts=True)
label2,count2 = np.unique(X_val.Label,return_counts=True)
label3,count3 = np.unique(X_test.Label,return_counts=True)

# DataFrames with 1 column for count of labels, label names in index
uni1 = pd.DataFrame(data=count1,index=labels,columns=['Count1'])
uni2 = pd.DataFrame(data=count2,index=labels,columns=['Count2'])
uni3 = pd.DataFrame(data=count3,index=labels,columns=['Count3'])

plt.figure(figsize=(20,6),dpi=200)
sns.set_style('darkgrid')

plt.subplot(131)
sns.barplot(data=uni1,x=uni1.index, y='Count1', hue=uni1.index, palette='icefire', legend=False).set_title('Class distribution in Training set',fontsize=15)
plt.xlabel('Labels',fontsize=12)
plt.ylabel('Count',fontsize=12)

plt.subplot(132)
sns.barplot(data=uni2,x=uni2.index, y='Count2', hue=uni1.index, palette='icefire', legend=False).set_title('Class distribution in validation set',fontsize=15)
plt.xlabel('Labels',fontsize=12)
plt.ylabel('Count',fontsize=12)


plt.subplot(133)
sns.barplot(data=uni3,x=uni3.index, y='Count3', hue=uni1.index, palette='icefire', legend=False).set_title('Class distribution in Testing set',fontsize=15)
plt.xlabel('Labels',fontsize=12)
plt.ylabel('Count',fontsize=12)

plt.show()
```



Deep Learning based Classifiers

In [11]:
```python
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.metrics import Precision, Recall
```

CNN

In [12]:
```python
# Load and preprocess data
IMG_SIZE = (32, 32)  # Adjust

def load_and_preprocess_image(filepath):
    image = tf.io.read_file(filepath)
    image = tf.image.decode_png(image, channels=3)
    image = tf.image.resize(image, IMG_SIZE)
    image = image / 255.0 # Normalize pixel values to [0, 1]
    return image
```

In [13]:
```python
base_dir = "/Users/renkasik/Desktop/Intro To Focus Areas/data/BreaKHis_v1"
```

In [14]:
```python
# Convert DataFrame columns to lists
train_filepaths = X_train['FilePath'].tolist()
train_labels = X_train['Label'].tolist()

val_filepaths = X_val['FilePath'].tolist()
val_labels = X_val['Label'].tolist()

test_filepaths = X_test['FilePath'].tolist()
test_labels = X_test['Label'].tolist()

# load images and labels as TensorFlow Datasets
def create_dataset(filepaths, labels):
    # Create TensorFlow datasets from paths and labels
    filepaths_ds = tf.data.Dataset.from_tensor_slices(filepaths)
    images_ds = filepaths_ds.map(load_and_preprocess_image, num_parallel_calls=tf.data.experimental.AUTOTUNE)
    labels_ds = tf.data.Dataset.from_tensor_slices(labels)

    # Zip the datasets together
    return tf.data.Dataset.zip((images_ds, labels_ds))
```

Data Augmentation

In [15]:
```python
# Define the augmentation layer
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),   # Randomly flip images horizontally
    tf.keras.layers.RandomRotation(0.2),        # Rotate images by up to 20%
    tf.keras.layers.RandomZoom(0.2),            # Zoom up to 20%
])

def augment(image, label):
    image = data_augmentation(image)
    return image, label
```

```
2024-11-12 11:32:33.979357: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical o
perations:  SSE4.1 SSE4.2
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
```

In [16]:
```python
i = 0
for filepath in train_filepaths:
    if not os.path.exists(filepath):
        print(f"File not found: {filepath}")
```

In [17]:
```python
BATCH_SIZE = 32  # Set the batch size according to memory capacity

# Shuffle, batch, and prefetch the datasets
train_ds = (create_dataset(train_filepaths, train_labels)
            .shuffle(buffer_size=len(train_filepaths))
            .map(augment, num_parallel_calls=tf.data.experimental.AUTOTUNE)
            .batch(BATCH_SIZE)
            .prefetch(tf.data.experimental.AUTOTUNE))
val_ds = create_dataset(val_filepaths, val_labels).batch(BATCH_SIZE).prefetch(tf.data.experimental.AUTOTUNE)
test_ds = create_dataset(test_filepaths, test_labels).batch(BATCH_SIZE).prefetch(tf.data.experimental.AUTOTUNE)
```

In [18]:
```python
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),  # Adjust input_shape to image dimensions, (3,3) is filter size, 3 is RGB color channels
    layers.MaxPooling2D((2, 2)), # reduces each dimension (height and width) by half, summarizing the main features
    layers.Conv2D(64, (3, 3), activation='relu'), # 64 filters
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'), # number of neurons in the layer
    layers.Dense(1, activation='sigmoid')
])
```

In [19]:
```python
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy', Precision(), Recall()])
```

In [20]:
```python
history = model.fit(train_ds,
                    validation_data=val_ds,
                    epochs=6)
```

```
Epoch 1/6
46/46 [==============================] - 4s 26ms/step - loss: 0.5783 - accuracy: 0.6937 - precision: 0.7178 - recall: 0.9164 - val_loss: 0.5217 - val_accuracy: 0.7796 - val_precision: 0.7928 - val_recall: 0.9213
Epoch 2/6
46/46 [==============================] - 3s 20ms/step - loss: 0.5039 - accuracy: 0.7775 - precision: 0.7923 - recall: 0.9184 - val_loss: 0.5413 - val_accuracy: 0.7220 - val_precision: 0.7172 - val_recall: 0.9861
Epoch 3/6
46/46 [==============================] - 3s 21ms/step - loss: 0.5412 - accuracy: 0.7088 - precision: 0.7141 - recall: 0.9642 - val_loss: 0.5286 - val_accuracy: 0.7348 - val_precision: 0.7270 - val_recall: 0.9861
Epoch 4/6
46/46 [==============================] - 3s 20ms/step - loss: 0.5049 - accuracy: 0.7747 - precision: 0.8047 - recall: 0.8896 - val_loss: 0.4584 - val_accuracy: 0.7923 - val_precision: 0.8159 - val_recall: 0.9028
Epoch 5/6
46/46 [==============================] - 3s 20ms/step - loss: 0.5007 - accuracy: 0.7850 - precision: 0.7962 - recall: 0.9254 - val_loss: 0.4605 - val_accuracy: 0.7859 - val_precision: 0.7992 - val_recall: 0.9213
Epoch 6/6
46/46 [==============================] - 3s 24ms/step - loss: 0.4942 - accuracy: 0.7837 - precision: 0.7875 - recall: 0.9403 - val_loss: 0.5723 - val_accuracy: 0.7476 - val_precision: 0.7370 - val_recall: 0.9861
```

In [21]:
```python
results = model.evaluate(test_ds)
metrics = ['Loss', 'Accuracy', 'Precision', 'Recall(Sensitivity)']
for i in range(len(results)):
    print(metrics[i] + ': ' + str(results[i]))
```

```
10/10 [==============================] - 1s 51ms/step - loss: 0.4983 - accuracy: 0.7853 - precision: 0.7709 - recall: 0.9815
Loss: 0.4982824921607971
Accuracy: 0.7852563858032227
Precision: 0.7709090709686279
Recall(Sensitivity): 0.9814814925193787
```

Fully connected NN with one hidden layer

In [22]:
```python
model_2 = models.Sequential([
    layers.Flatten(input_shape=(32, 32, 3)),  # fully connected -> flatten from 2D to 1D
    layers.Dense(64, activation='relu'),  # hidden layer -> learning features from the input data
    layers.Dense(1, activation='sigmoid')
])
```

In [23]:
```python
model_2.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy', Precision(), Recall()])
```

In [24]:
```python
history_2 = model_2.fit(train_ds,
                    validation_data=val_ds,
                    epochs=10)
```

```
Epoch 1/10
46/46 [==============================] - 3s 16ms/step - loss: 0.7119 - accuracy: 0.6525 - precision_1: 0.7153 - recall_1: 0.8249 - val_loss: 0.6900 - val_accuracy: 0.5272 - val_precision_1: 0.9146 - val_recall_1: 0.3472
Epoch 2/10
46/46 [==============================] - 3s 12ms/step - loss: 0.5366 - accuracy: 0.7445 - precision_1: 0.7653 - recall_1: 0.9085 - val_loss: 0.5109 - val_accuracy: 0.6933 - val_precision_1: 0.6923 - val_recall_1: 1.0000
Epoch 3/10
46/46 [==============================] - 3s 13ms/step - loss: 0.5553 - accuracy: 0.7184 - precision_1: 0.7617 - recall_1: 0.8617 - val_loss: 0.4805 - val_accuracy: 0.7955 - val_precision_1: 0.8016 - val_recall_1: 0.9352
Epoch 4/10
46/46 [==============================] - 3s 12ms/step - loss: 0.5075 - accuracy: 0.7788 - precision_1: 0.7998 - recall_1: 0.9065 - val_loss: 0.5351 - val_accuracy: 0.6965 - val_precision_1: 0.6945 - val_recall_1: 1.0000
Epoch 5/10
46/46 [==============================] - 2s 12ms/step - loss: 0.5061 - accuracy: 0.7624 - precision_1: 0.7955 - recall_1: 0.8826 - val_loss: 0.5112 - val_accuracy: 0.7284 - val_precision_1: 0.8743 - val_recall_1: 0.7083
Epoch 6/10
46/46 [==============================] - 3s 12ms/step - loss: 0.5130 - accuracy: 0.7706 - precision_1: 0.8020 - recall_1: 0.8866 - val_loss: 0.4780 - val_accuracy: 0.8083 - val_precision_1: 0.8277 - val_recall_1: 0.9120
Epoch 7/10
46/46 [==============================] - 3s 12ms/step - loss: 0.5185 - accuracy: 0.7596 - precision_1: 0.7942 - recall_1: 0.8796 - val_loss: 0.5722 - val_accuracy: 0.6550 - val_precision_1: 0.8913 - val_recall_1: 0.5694
Epoch 8/10
46/46 [==============================] - 2s 12ms/step - loss: 0.5111 - accuracy: 0.7624 - precision_1: 0.8015 - recall_1: 0.8716 - val_loss: 0.4906 - val_accuracy: 0.7955 - val_precision_1: 0.7754 - val_recall_1: 0.9907
Epoch 9/10
46/46 [==============================] - 2s 11ms/step - loss: 0.5379 - accuracy: 0.7445 - precision_1: 0.7944 - recall_1: 0.8498 - val_loss: 0.5651 - val_accuracy: 0.6550 - val_precision_1: 0.8857 - val_recall_1: 0.5741
Epoch 10/10
46/46 [==============================] - 2s 12ms/step - loss: 0.5035 - accuracy: 0.7685 - precision_1: 0.7977 - recall_1: 0.8905 - val_loss: 0.4850 - val_accuracy: 0.7987 - val_precision_1: 0.8558 - val_recall_1: 0.8519
```

In [25]:
```python
results_2 = model_2.evaluate(test_ds)
metrics = ['Loss', 'Accuracy', 'Precision', 'Recall(Sensitivity)']
for i in range(len(results_2)):
    print(metrics[i] + ': ' + str(results_2[i]))
```

```
10/10 [==============================] - 0s 41ms/step - loss: 0.4484 - accuracy: 0.8333 - precision_1: 0.8761 - recall_1: 0.8843
Loss: 0.4483715891838074
Accuracy: 0.8333333134651184
Precision: 0.8761467933654785
Recall(Sensitivity): 0.8842592835426331
```

ResNet-50

In [26]:
```python
from tensorflow.keras.applications import ResNet50
```

In [27]:
```python
IMG_SIZE_resnet = (224, 224) #ResNet50 input size requirement

def load_and_preprocess_image_resnet(filepath):
    image = tf.io.read_file(filepath)
    image = tf.image.decode_png(image, channels=3)
    image = tf.image.resize(image, IMG_SIZE_resnet)
    image = tf.keras.applications.resnet50.preprocess_input(image) #ResNet preprocessing
    return image
```

In [28]:
```python
def create_dataset_resnet(filepaths, labels):
    # Create TensorFlow datasets from paths and labels for ResNet
    filepaths_ds = tf.data.Dataset.from_tensor_slices(filepaths)
    images_ds = filepaths_ds.map(load_and_preprocess_image_resnet, num_parallel_calls=tf.data.experimental.AUTOTUNE)
    labels_ds = tf.data.Dataset.from_tensor_slices(labels)

    # Zip the datasets together
    return tf.data.Dataset.zip((images_ds, labels_ds))
```

In [29]:
```python
# Shuffle, batch, and prefetch the datasets
train_ds_resnet = (create_dataset_resnet(train_filepaths, train_labels)
                   .shuffle(buffer_size=len(train_filepaths))
                   .map(augment, num_parallel_calls=tf.data.experimental.AUTOTUNE)
                   .batch(BATCH_SIZE)
                   .prefetch(tf.data.experimental.AUTOTUNE))
val_ds_resnet = create_dataset_resnet(val_filepaths, val_labels).batch(BATCH_SIZE).prefetch(tf.data.experimental.AUTOTUNE)
test_ds_resnet = create_dataset_resnet(test_filepaths, test_labels).batch(BATCH_SIZE).prefetch(tf.data.experimental.AUTOTUNE)
```

In [30]:
```python
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3)) #without the top classification layer to add custom classification layers later
base_model.trainable = False  #freeze base model to avoid fine tuning
```

In [31]:
```python
model_3 = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(), #reduce output to single vector
    layers.Dense(1, activation='sigmoid')
])
```

In [32]:
```python
model_3.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy', Precision(), Recall()])
```

In [33]:
```python
history_3 = model_3.fit(train_ds_resnet,
                    validation_data=val_ds_resnet,
                    epochs=10)
```

```
Epoch 1/10
46/46 [==============================] - 95s 2s/step - loss: 0.6119 - accuracy: 0.6999 - precision_2: 0.7371 - recall_2: 0.8786 - val_loss: 0.4488 - val_accuracy: 0.7891 - val_precision_2: 0.8788 - val_recall_2: 0.8056
Epoch 2/10
46/46 [==============================] - 86s 2s/step - loss: 0.3784 - accuracy: 0.8386 - precision_2: 0.8407 - recall_2: 0.9453 - val_loss: 0.3430 - val_accuracy: 0.8530 - val_precision_2: 0.8728 - val_recall_2: 0.9213
Epoch 3/10
46/46 [==============================] - 84s 2s/step - loss: 0.3147 - accuracy: 0.8750 - precision_2: 0.8807 - recall_2: 0.9473 - val_loss: 0.3391 - val_accuracy: 0.8498 - val_precision_2: 0.9378 - val_recall_2: 0.8380
Epoch 4/10
46/46 [==============================] - 85s 2s/step - loss: 0.2865 - accuracy: 0.8846 - precision_2: 0.8915 - recall_2: 0.9483 - val_loss: 0.2872 - val_accuracy: 0.8850 - val_precision_2: 0.9286 - val_recall_2: 0.9028
Epoch 5/10
46/46 [==============================] - 85s 2s/step - loss: 0.2648 - accuracy: 0.8984 - precision_2: 0.9039 - recall_2: 0.9542 - val_loss: 0.2759 - val_accuracy: 0.8978 - val_precision_2: 0.9510 - val_recall_2: 0.8981
Epoch 6/10
46/46 [==============================] - 84s 2s/step - loss: 0.2440 - accuracy: 0.9121 - precision_2: 0.9237 - recall_2: 0.9512 - val_loss: 0.2459 - val_accuracy: 0.9042 - val_precision_2: 0.9266 - val_recall_2: 0.9352
Epoch 7/10
46/46 [==============================] - 84s 2s/step - loss: 0.2319 - accuracy: 0.9141 - precision_2: 0.9183 - recall_2: 0.9612 - val_loss: 0.2345 - val_accuracy: 0.9137 - val_precision_2: 0.9276 - val_recall_2: 0.9491
Epoch 8/10
46/46 [==============================] - 85s 2s/step - loss: 0.2174 - accuracy: 0.9114 - precision_2: 0.9203 - recall_2: 0.9542 - val_loss: 0.2265 - val_accuracy: 0.9201 - val_precision_2: 0.9484 - val_recall_2: 0.9352
Epoch 9/10
46/46 [==============================] - 86s 2s/step - loss: 0.2076 - accuracy: 0.9327 - precision_2: 0.9365 - recall_2: 0.9682 - val_loss: 0.2194 - val_accuracy: 0.9233 - val_precision_2: 0.9615 - val_recall_2: 0.9259
Epoch 10/10
46/46 [==============================] - 85s 2s/step - loss: 0.2031 - accuracy: 0.9203 - precision_2: 0.9303 - recall_2: 0.9562 - val_loss: 0.2139 - val_accuracy: 0.9233 - val_precision_2: 0.9528 - val_recall_2: 0.9352
```

In [ ]:
```python
#fine tuning
base_model.trainable = True
model_3.compile(optimizer=tf.keras.optimizers.Adam(1e-5),  #lower learning rate
              loss='binary_crossentropy',
              metrics=['accuracy', Precision(), Recall()])

history_3 = model_3.fit(train_ds_resnet,
                    validation_data=val_ds_resnet,
                    epochs=5)
```

```
Epoch 1/5
46/46 [==============================] - 299s 6s/step - loss: 0.7331 - accuracy: 0.6841 - precision_3: 0.9595 - recall_3: 0.5662 - val_loss: 0.3274 - val_accuracy: 0.8498 - val_precision_3: 0.8238 - val_recall_3: 0.9954
Epoch 2/5
46/46 [==============================] - 291s 6s/step - loss: 0.2873 - accuracy: 0.8832 - precision_3: 0.9523 - recall_3: 0.8746 - val_loss: 0.2820 - val_accuracy: 0.8786 - val_precision_3: 0.8560 - val_recall_3: 0.9907
Epoch 3/5
46/46 [==============================] - 289s 6s/step - loss: 0.1962 - accuracy: 0.9245 - precision_3: 0.9497 - recall_3: 0.9403 - val_loss: 0.2304 - val_accuracy: 0.9010 - val_precision_3: 0.8838 - val_recall_3: 0.9861
Epoch 4/5
46/46 [==============================] - 285s 6s/step - loss: 0.1625 - accuracy: 0.9437 - precision_3: 0.9629 - recall_3: 0.9552 - val_loss: 0.1823 - val_accuracy: 0.9297 - val_precision_3: 0.9181 - val_recall_3: 0.9861
Epoch 5/5
 2/46 [>.............................] - ETA: 4:18 - loss: 0.2091 - accuracy: 0.9219 - precision_3: 0.9500 - recall_3: 0.9268
```

In [ ]:
```python
results_3 = model_3.evaluate(test_ds_resnet)
metrics = ['Loss', 'Accuracy', 'Precision', 'Recall(Sensitivity)']
for i in range(len(results_3)):
    print(metrics[i] + ': ' + str(results_3[i]))
```

```
10/10 [==============================] - 14s 1s/step - loss: 0.1844 - accuracy: 0.9199 - precision_3: 0.9207 - recall_3: 0.9676
Loss: 0.18444310128688812
Accuracy: 0.9198718070983887
Precision: 0.9207048416137695
Recall(Sensitivity): 0.9675925970077515
```

Plots

Metric Comparison

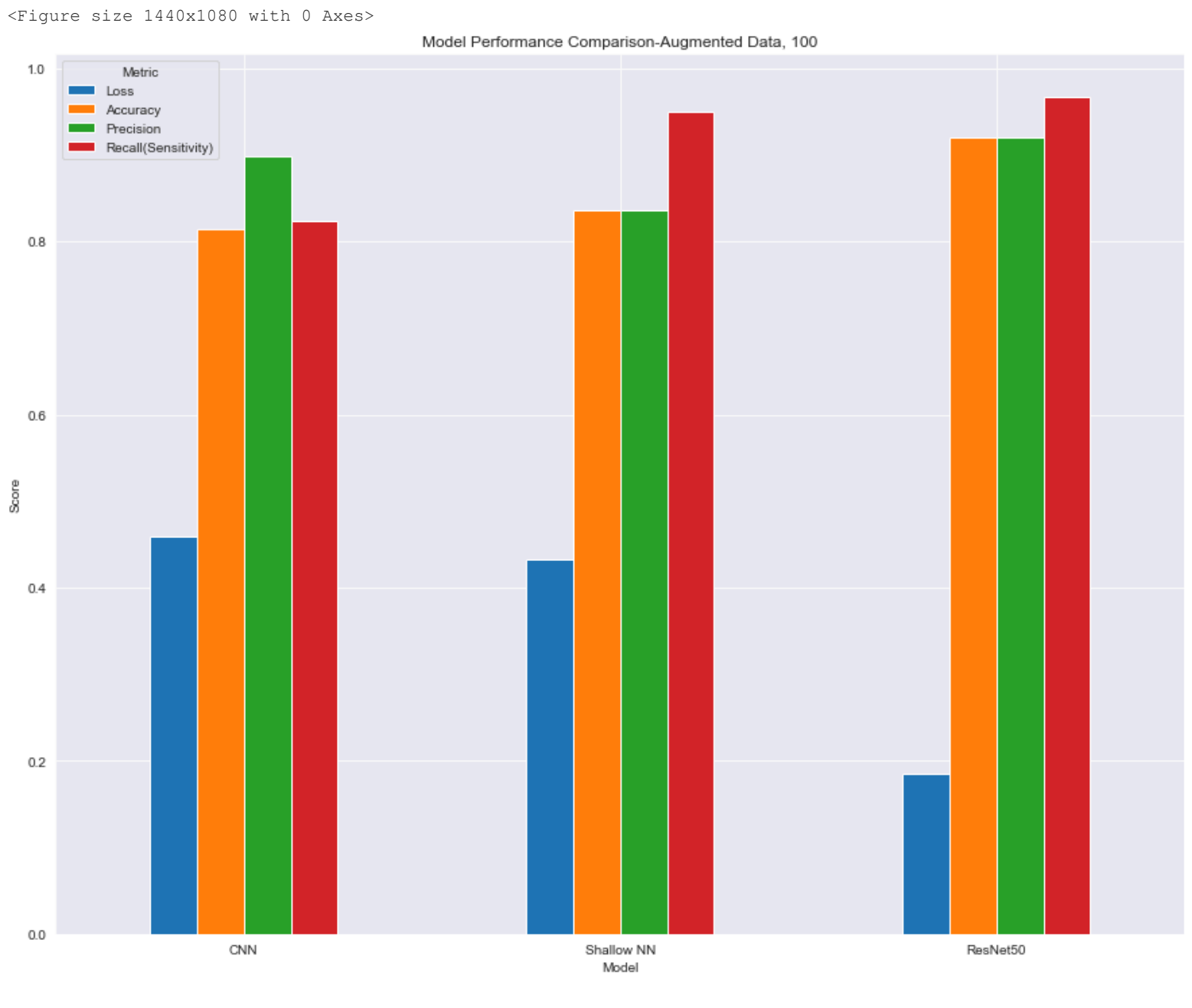
In [ ]:
```python
results = np.array(results)
results_2 = np.array(results_2)
results_3 = np.array(results_3)
results_df = pd.DataFrame({'CNN': results, 'Shallow NN': results_2, 'ResNet50': results_3})

results_df = results_df.T

results_df.columns = ['Loss', 'Accuracy', 'Precision', 'Recall(Sensitivity)']
results_df
```

Out[ ]:

	Loss	Accuracy	Precision	Recall(Sensitivity)
CNN	0.458682	0.814103	0.898990	0.824074
Shallow NN	0.432735	0.836538	0.836735	0.949074
ResNet50	0.184443	0.919872	0.920705	0.967593

In [ ]:
```python
plt.figure(figsize=(20, 15))

results_df.plot(kind='bar', rot=0, figsize=(15,12))
plt.title('Model Performance Comparison-Augmented Data, 100')
plt.ylabel('Score')
plt.xlabel('Model')

# Show the plot
plt.legend(title='Metric')
plt.show()
```

```
<Figure size 1440x1080 with 0 Axes>
```



ROC Curves

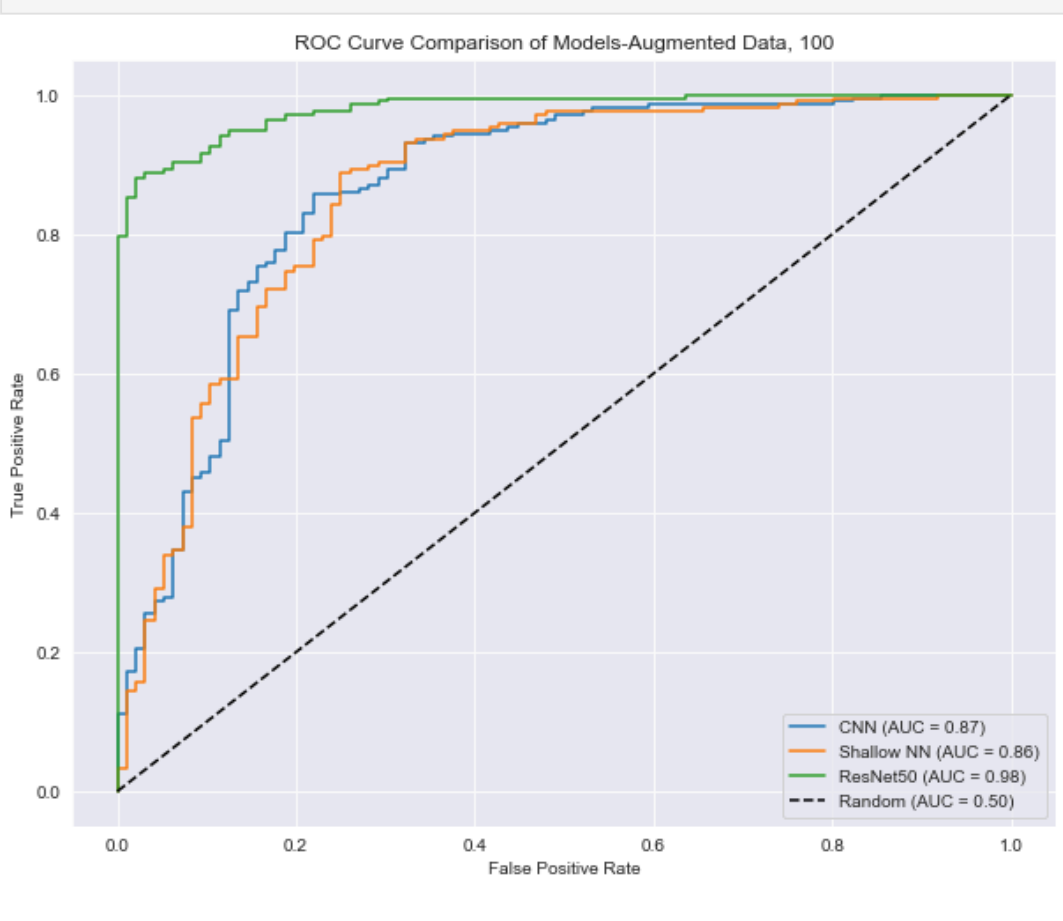
In [ ]:
```python
from sklearn.metrics import roc_curve, auc
```

In [ ]:
```python
X_test_images = np.array([load_and_preprocess_image(fp) for fp in X_test['FilePath']])
X_test_images_resnet = np.array([load_and_preprocess_image_resnet(fp) for fp in X_test['FilePath']])

probabilities_CNN = model.predict(X_test_images)
probabilities_ShallowNN = model_2.predict(X_test_images)
probabilities_ResNet50 = model_3.predict(X_test_images_resnet)
```

```
10/10 [==============================] - 0s 4ms/step
10/10 [==============================] - 0s 747us/step
10/10 [==============================] - 14s 1s/step
```

In [ ]:
```python
fpr_CNN, tpr_CNN, _ = roc_curve(test_labels, probabilities_CNN)
fpr_ShallowNN, tpr_ShallowNN, _ = roc_curve(test_labels, probabilities_ShallowNN)
fpr_ResNet50, tpr_ResNet50, _ = roc_curve(test_labels, probabilities_ResNet50)

auc_CNN = auc(fpr_CNN, tpr_CNN)
auc_ShallowNN = auc(fpr_ShallowNN, tpr_ShallowNN)
auc_ResNet50 = auc(fpr_ResNet50, tpr_ResNet50)
```

In [ ]:
```python
plt.figure(figsize=(10, 8))


plt.plot(fpr_CNN, tpr_CNN, label=f'CNN (AUC = {auc_CNN:.2f})')
plt.plot(fpr_ShallowNN, tpr_ShallowNN, label=f'Shallow NN (AUC = {auc_ShallowNN:.2f})')
plt.plot(fpr_ResNet50, tpr_ResNet50, label=f'ResNet50 (AUC = {auc_ResNet50:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random (AUC = 0.50)')


plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve Comparison of Models-Augmented Data, 100')
plt.legend(loc='lower right')
plt.show()
```



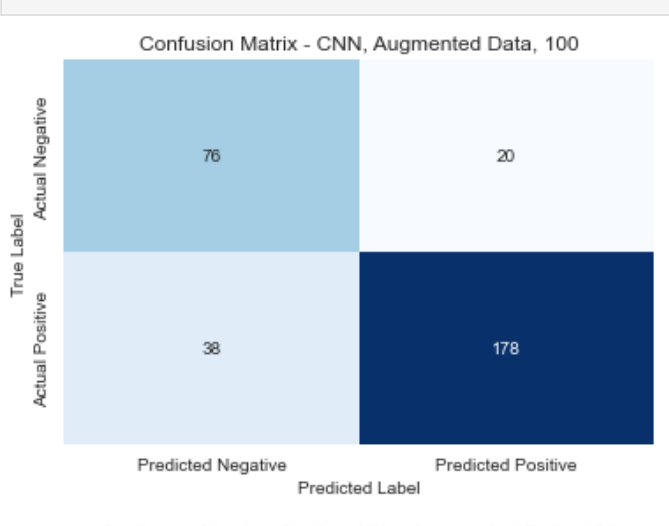
Confusion Matrix

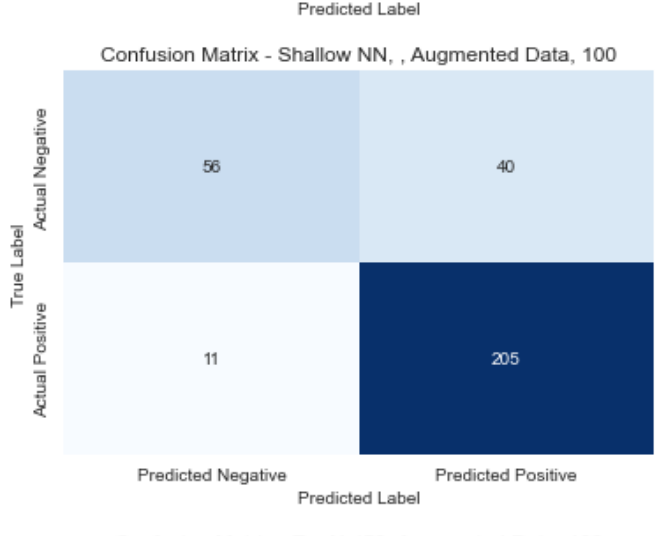
In [ ]:
```python
from sklearn.metrics import confusion_matrix
```

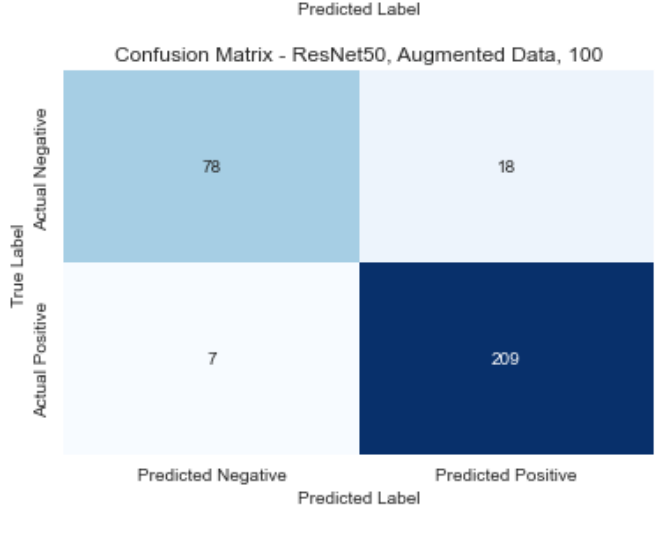
In [ ]:
```python
#probabilities to binary predictions
pred_CNN = (probabilities_CNN > 0.5).astype(int)
pred_ShallowNN = (probabilities_ShallowNN > 0.5).astype(int)
pred_ResNet50 = (probabilities_ResNet50 > 0.5).astype(int)
```

In [ ]:
```python
cm_CNN = confusion_matrix(test_labels, pred_CNN)
cm_ShallowNN = confusion_matrix(test_labels, pred_ShallowNN)
cm_ResNet50 = confusion_matrix(test_labels, pred_ResNet50)
```

In [ ]:
```python
def plot_confusion_matrix(cm, title):
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
                xticklabels=['Predicted Negative', 'Predicted Positive'],
                yticklabels=['Actual Negative', 'Actual Positive'])
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.title(title)
    plt.show()

plot_confusion_matrix(cm_CNN, "Confusion Matrix - CNN, Augmented Data, 100")
plot_confusion_matrix(cm_ShallowNN, "Confusion Matrix - Shallow NN, , Augmented Data, 100")
plot_confusion_matrix(cm_ResNet50, "Confusion Matrix - ResNet50, Augmented Data, 100")
```





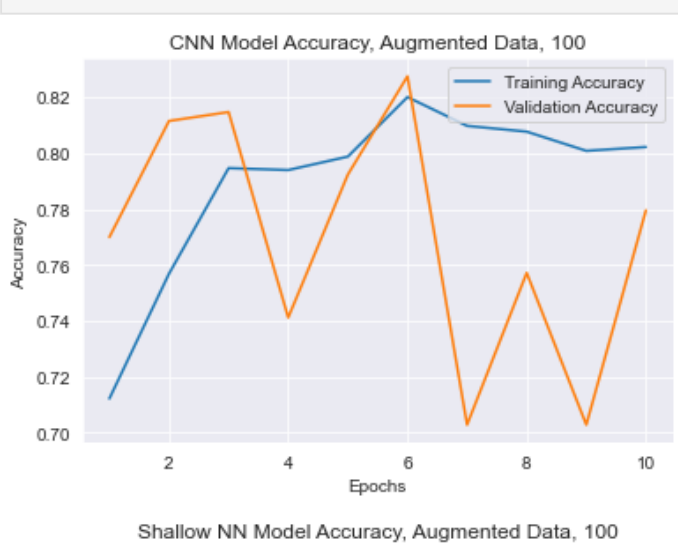


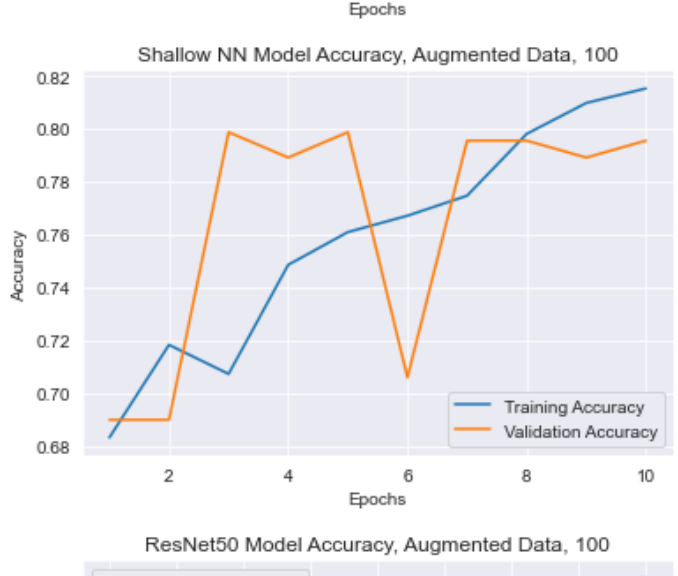
Learning Curves

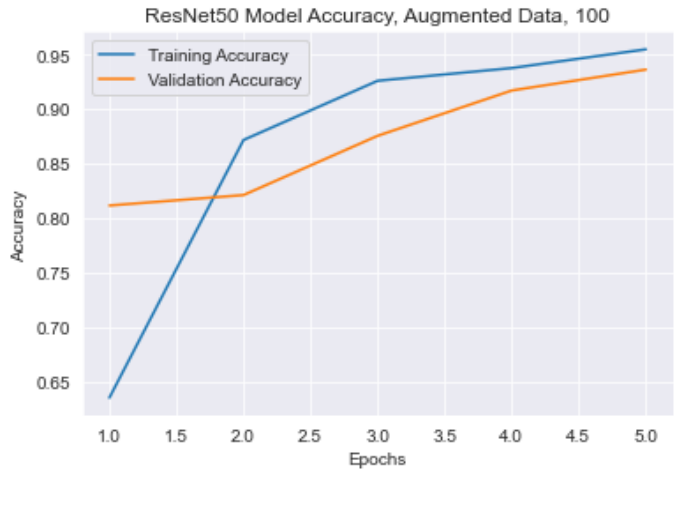
Training/Validation Accuracy

In [ ]:
```python
def plot_accuracy(history, title):
    epochs = range(1, len(history.history['accuracy']) + 1)
    plt.plot(epochs, history.history['accuracy'], label='Training Accuracy')
    plt.plot(epochs, history.history['val_accuracy'], label='Validation Accuracy')
    plt.title(title)
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()
```

In [ ]:
```python
plot_accuracy(history, 'CNN Model Accuracy, Augmented Data, 100')
plot_accuracy(history_2, 'Shallow NN Model Accuracy, Augmented Data, 100')
plot_accuracy(history_3, 'ResNet50 Model Accuracy, Augmented Data, 100')
```







Training/Validation Loss

In [ ]:
```python
def plot_loss(history, title):
    epochs = range(1, len(history.history['loss']) + 1)
    plt.plot(epochs, history.history['loss'], label='Training Loss')
    plt.plot(epochs, history.history['val_loss'], label='Validation Loss')
    plt.title(title)
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()
```

In [ ]:
```python
plot_loss(history, 'CNN Model Loss, Augmented Data, 100')
plot_loss(history_2, 'Shallow NN Model Loss, Augmented Data, 100')
plot_loss(history_3, 'ResNet50 Model Loss, Augmented Data, 100')
```